

🌱 一、延时与事件控制

行为级写法 (仿真专用)	可综合 RTL 写法	说明
<code>#10 a = b;</code>	不可综合 ❌	延时 10 时间单位是仿真语义，硬件无此机制
<code>@(posedge clk)</code> 仅用于 testbench 驱动	<code>always @(posedge clk)</code>	RTL 时序块需放在 always 头部
<code>wait (sig == 1)</code>	不建议综合 (取决于工具)	推荐用状态机显式控制

✅ 要点:

RTL 中只能使用时钟边沿或异步复位事件触发，不允许使用 `#` 延时。

⚙️ 二、变量赋值方式

行为级写法	可综合 RTL 写法	说明
<code>a = b;</code> (阻塞赋值) 用于时序逻辑	❌ 容易导致竞态	时序逻辑用 <code><=</code> 非阻塞赋值
<code><=</code> (非阻塞赋值) 用于组合逻辑	❌ 不推荐	组合逻辑用 <code>=</code>
<code>always @(posedge clk) a = b;</code>	✅ 改为 <code>a <= b;</code>	时序块标准写法
<code>always @(*) a <= b & c;</code>	✅ 改为 <code>a = b & c;</code>	组合块标准写法

◆ 规则:

- “时序逻辑 (寄存器) 用 **非阻塞** `<=`”
- “组合逻辑 (纯逻辑) 用 **阻塞** `=`”

💬 三、循环与流程控制

行为级写法	可综合 RTL 写法	说明
<code>forever begin ... end</code>	❌ 仅仿真循环	
<code>repeat(10) ...</code>	❌ 仿真控制循环	
<code>for (i=0; i<8; i=i+1)</code>	✅ 可综合 (循环次数固定)	
<code>while(a < b)</code>	❌ 通常不可综合 (不定循环次数)	

✅ 原则:

只允许**固定循环次数**的 `for`; 其它循环语句仿真可用，硬件不可实现。

🧠 四、任务与函数

行为级写法	可综合 RTL 写法	说明	📄
<code>task delay_10; #10; endtask</code>	❌ 不可综合 (含延时)		
<code>function automatic int sum; ... endfunction</code>	✅ 可综合 (无延时/系统任务)		
<code>\$display</code> , <code>\$stop</code> , <code>\$finish</code> , <code>\$random</code>	❌ 仿真系统任务, 不可综合		
自定义函数中只含逻辑运算	✅ 可综合		

📦 五、初始块 (`initial`)

行为级写法	可综合 RTL 写法	说明
<code>initial begin a = 1; b = 2; end</code>	❌ ASIC 不可综合; 部分 FPGA 可初始化寄存器	
在 testbench 中使用 <code>initial</code>	✅ 仿真驱动 OK	
用复位逻辑初始化寄存器	✅ 推荐做法	

✅ 建议:

硬件上电初始化应通过**同步或异步复位**实现, 不依赖 `initial`。

🌿 六、文件 I/O 与系统任务

行为级写法	可综合 RTL 写法	说明	📄
<code>\$display</code> , <code>\$write</code> , <code>\$strobe</code> , <code>\$monitor</code>	❌ 仿真专用打印		
<code>\$readmemh</code> , <code>\$readmemb</code>	⚠️ FPGA 有限支持; ASIC 不可综合		
<code>\$stop</code> , <code>\$finish</code>	❌ 仿真停止语句		
<code>\$time</code> , <code>\$realtime</code>	❌ 仿真时间函数		
<code>\$fopen</code> , <code>\$fwrite</code>	❌ 仿真专用文件操作		

✅ 要点:

以上语句**都是 testbench 工具行为**, 综合时会被忽略或报错。

🔧 七、case 与 if 的写法差异

行为级写法	可综合 RTL 写法	说明	📄
<code>casex</code> 、 <code>casez</code> 全通配	⚠️ 可综合, 但掩盖 <code>x/z</code> , 要小心		
<code>case(sel)</code> 不写 <code>default</code>	⚠️ 可能综合出锁存器		
<code>if</code> 少写 <code>else</code>	⚠️ 同样可能综合锁存器		
完整 <code>if-else</code> 或带 <code>default</code> 的 <code>case</code>	✅ 推荐写法		

🕒 八、仿真时间 vs 硬件时间

行为级语法	RTL 等价	说明	📄
<code>#10</code> 延时 10ns	✅ 无等价，硬件无法延时	延时只能通过计数器实现	
<code>\$time</code> , <code>\$realtime</code>	✅ 用于调试	不存在于硬件	
<code>@(posedge clk)</code>	✅ 时序触发	可综合	
<code>@(negedge rst_n)</code>	✅ 异步复位触发	可综合（异步复位）	

🎲 九、随机与延迟信号

行为级写法	可综合 RTL 写法	说明	📄
<code>a = \$random;</code>	❌ 仿真随机数，不可综合		
延迟赋值: <code>a = #5 b;</code>	❌ 仿真延时		
计数器分频延时	✅ 可综合		
状态机延时控制	✅ 可综合		

✅ 十、整体判断法（非常实用）

检查问题	是否可综合?	说明	📄
用了 <code>#延时</code> / <code>\$display</code> 吗?	❌	仿真语法	
<code>always</code> 是否用时钟触发?	✅	RTL	
逻辑中是否全赋值（无锁存）?	✅	RTL	
是否依赖文件 I/O?	❌	仿真语义	
是否使用固定次数 for 循环?	✅	RTL	
是否用 <code>\$random</code> ?	❌	仿真语义	
是否用 <code>initial</code> 初始化?	⚠️ 仅限 FPGA	建议复位初始化	

NVboard 仿真与 gtkwave 仿真对比

🔧 三、verilated.h 具体提供了什么？

打开 `/usr/local/share/verilator/include/verilated.h` 可以看到，它定义了：

功能	示例	用途
仿真控制类	<code>Verilated::commandArgs()</code>	传递命令行参数
时间控制类型	<code>vluint64_t</code>	仿真时间计数器
全局时间接口	<code>double sc_time_stamp()</code>	<code>\$time</code> <code>\$realtime</code> 支持
收尾函数	<code>Verilated::gotFinish()</code> 、 <code>dut->final()</code>	优雅退出仿真
trace 打开/关闭	<code>Verilated::traceEverOn(true)</code>	启用波形追踪
Verilator runtime 控制	版本号、调试选项等	调试与日志输出

没有它，你的 C++ 编译器根本不知道这些类和函数的存在，自然也无法链接正确。

NVboard 框架

🔧 核心逻辑结构

cpp

复制代码

🔍 解释

```
#include "Vlight.h"
#include "nvboard.h"

static Vlight *dut = new Vlight;

void single_cycle() {
    dut->clk = 0; dut->eval();
    dut->clk = 1; dut->eval();
}

int main() {
    nvboard_bind_all_pins(dut);
    nvboard_init();

    while (true) {
        nvboard_update(); // 从GUI更新输入/输出状态
        single_cycle();   // 推进一个逻辑周期
    }

    nvboard_quit();
}
```

Gtkwave 框架

核心逻辑结构

cpp

复制代码

解释

```
#include <verilated.h>
#include <verilated_vcd_c.h> // 或 <verilated_fst_c.h>
#include "Vlight.h"

static Vlight *dut = new Vlight;
static VerilatedVcdC *tfp = new VerilatedVcdC;
static vluint64_t main_time = 0;
double sc_time_stamp() { return main_time; }

void tick() {
    dut->clk = 0; dut->eval(); tfp->dump(main_time); main_time += 5;
    dut->clk = 1; dut->eval(); tfp->dump(main_time); main_time += 5;
}

int main() {
    Verilated::traceEverOn(true);
    dut->trace(tfp, 99);
    tfp->open("light.vcd");

    for (int i = 0; i < 100; i++) tick();

    tfp->close();
    delete tfp;
    delete dut;
}
```

↓

关键点

项目	内容	📄
目标	输出 <code>.vcd</code> 或 <code>.fst</code> 波形文件供 GTKWave 分析	
时间控制	需要显式 <code>main_time</code> 并定义 <code>sc_time_stamp()</code>	
波形文件	✅ 必须设置 <code>trace</code> 并调用 <code>tfp->dump()</code>	
时钟产生方式	<code>tick()</code> 函数中人工翻转 <code>clk</code> + 推进时间	
典型接口	<code>Verilated::traceEverOn()</code> / <code>tfp->open()</code> / <code>tfp->dump()</code> / <code>tfp->close()</code>	
是否需要 <code><verilated.h></code>	✅ 必须 (包含 Verilator API 定义)	
运行方式	离线执行、结束后再查看波形	
仿真速度	较快 (无 GUI, 纯逻辑求值 + 文件IO)	
适合用途	精确逻辑验证 / 时序调试 / 输出波形分析	

verilog 是否可综合语句

类别	可综合的语法（能变成硬件电路）		不可综合的语法（仅用于仿真）
初始化和赋值	<code>initial</code> （仅限FPGA，用于给寄存器赋初值，会上电后初始化，但ASIC中不可综合）		<code>initial</code> （在ASIC中不可综合，用于Testbench初始化）
	<code>always</code> （是关键的可综合块）		<code>always</code> （如果敏感列表或内部逻辑不满足可综合要求，则不可综合）
	阻塞赋值 <code>=</code> （在 <code>always @(*)</code> 中用于组合逻辑）		
	非阻塞赋值 <code><=</code> （在 <code>always @(posedge clk)</code> 中用于时序逻辑）		
进程控制	<code>if...else</code> , <code>case</code> , <code>casex</code> , <code>casez</code> （在 <code>always</code> 块内）		<code>for</code> （谨慎使用，只有当循环次数在编译时固定且可展开时才可综合）
	<code>for</code> （循环展开，用于简化代码，综合后是重复的硬件）		<code>while</code> , <code>forever</code> , <code>repeat</code> （仿真时动态循环，无法综合为固定硬件）
延时控制	无		<code>#5</code> （延时控制，物理电路没有绝对的延时单元）
系统任务/函数	无		<code>\$display</code> , <code>\$monitor</code> , <code>\$finish</code> , <code>\$stop</code> （输出到屏幕，硬件无法实现）
文件操作	无		<code>\$readmemh</code> , <code>\$readmemb</code> （从文件读取数据，仅用于初始化存储器，综合时会被忽略或仅部分支持）
值表示	所有 <code>parameter</code> , <code>localparam</code> , 常量		<code>x</code> （不定态）， <code>z</code> （高阻态）（在综合时会被处理为确定值0或1）
类别	语句	可综合性	说明
过程语句	<code>initial</code>	△	FPGA部分可综合（用于初始化），ASIC不可综合
	<code>always</code>	√	完全可综合，但敏感列表和内部逻辑必须符合可综合要求
语句块	串行语句块 <code>begin-end</code>	√	完全可综合
	并行语句块 <code>fork-join</code>	×	不可综合，仅用于仿真
赋值语句	过程连续赋值 <code>assign</code>	×	不可综合，指 <code>assign</code> 在过程块内使用的情况
	过程赋值语句 <code>=</code> , <code><=</code>	√	完全可综合，但必须正确使用（组合用 <code>=</code> ，时序用 <code><=</code> ）
条件语句	<code>if-else</code>	√	完全可综合
	<code>case</code> , <code>casez</code> , <code>casex</code>	√	完全可综合，但 <code>casex</code> 要谨慎使用
循环语句	<code>forever</code>	×	不可综合，仅用于仿真
	<code>repeat</code>	×	不可综合，仅用于仿真
	<code>while</code>	×	不可综合，仅用于仿真
	<code>for</code>	△	条件可综合（循环边界必须在编译时确定）
编译向导语句	<code>'define</code>	√	完全可综合，文本替换
	<code>'include</code>	√	完全可综合，文件包含
	<code>'ifdef</code> , <code>'else</code> , <code>'endif</code>	√	完全可综合，条件编译